

Bachelorarbeit

---

# Ansatz zur Optimierung einer CI-Pipeline mit reproduzierbaren lokalen Builds

---

Ruben Leander Rosenmeyer

Gutachter: Prof. Dr. Peter Thiemann

Betreuer: Dr. Michael Sperber

Albert-Ludwigs-Universität Freiburg

Technische Fakultät

Institut für Informatik

13. Juli 2026

**Bearbeitungszeit**

13. 04. 2026 – 13. 07. 2026

**Gutachter**

Prof. Dr. Peter Thiemann

**Betreuer**

Dr. Michael Sperber

# ERKLÄRUNG

Hiermit erkläre ich, dass ich diese Abschlussarbeit selbständig verfasst habe, keine anderen als die angegebenen Quellen/Hilfsmittel verwendet habe und alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten Schriften entnommen wurden, als solche kenntlich gemacht habe.

Darüber hinaus erkläre ich, dass diese Abschlussarbeit nicht, auch nicht auszugsweise, bereits für eine andere Prüfung angefertigt wurde.

---

Ort, Datum

---

Unterschrift

# Zusammenfassung

Diese Bachelorarbeit beschäftigt sich mit einem Ansatz, der die Gesamtlaufzeit einer Testsuite eines Softwareprojektes reduzieren soll. Das soll erreicht werden, indem vermieden wird, dass Testfälle, die bereits für den aktuellen Stand der Softwareprojekts evaluiert wurden, erneut ausgeführt werden. Dieser Ansatz bewegt sich in einem Szenario, bei dem eine unbestimmte Teilmenge aller auszuführenden Testfälle bereits durchlaufen wurde. Beispielsweise kann es sich dabei um einen Build-Server handeln, der einen Commit von einem Entwickler erhält, welcher bereits bei sich lokal eine Teilmenge der Tests laufen lassen hat.

Hierfür werden im Rahmen dieser Bachelorarbeit die Anforderungen für ein Tooling, welches den oben beschriebenen Ansatz implementiert, beschrieben und diskutiert. Außerdem wird der allgemeine Aufbau von Testing-Frameworks und ihre möglichen Ausbagerformate von Testläufen sowie die resultierenden Konsequenzen für die frameworkunabhängigkeit des Toolings näher beleuchtet. Dazu wird ein Datensatz untersucht, der einige Implementierungen eines simplen Testablaufs sowie die resultierende Ausgabe in ausgewählten Frameworks in unterschiedlichen Programmiersprachen beinhaltet.

Begleitet wird diese Arbeit von einer Proof-of-Concept-Umsetzung des hier vorgestellten Tools, einer Demo dieses Tools in einer exemplarischen Umgebung, sowie dem genannten Datensatz.

# Inhaltsverzeichnis

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>Zusammenfassung</b>                                       | <b>ii</b> |
| <b>1 Einleitung</b>                                          | <b>1</b>  |
| 1.1 Problemstellung . . . . .                                | 1         |
| 1.2 Zielsetzung . . . . .                                    | 2         |
| <b>2 Anforderungen</b>                                       | <b>3</b>  |
| 2.1 Anforderungen an das Tool . . . . .                      | 3         |
| 2.2 Anforderungen an ein Testausgabeformat . . . . .         | 3         |
| 2.3 Wie Output entsteht . . . . .                            | 5         |
| 2.4 git notes . . . . .                                      | 5         |
| <b>3 Vergleich von Testausgabeformaten</b>                   | <b>6</b>  |
| 3.1 Evaluation der häufigsten Formate . . . . .              | 7         |
| 3.1.1 JUnit XML . . . . .                                    | 7         |
| 3.1.2 Test Anything Protocol (TAP) . . . . .                 | 8         |
| 3.1.3 Open Test Reporting (OTR) . . . . .                    | 10        |
| 3.1.4 Common Test Report Format (CTRF) . . . . .             | 11        |
| 3.1.5 Andere Formate . . . . .                               | 12        |
| 3.2 Fazit . . . . .                                          | 12        |
| <b>4 Umsetzung</b>                                           | <b>13</b> |
| 4.1 Anforderungen an ein Projekt . . . . .                   | 13        |
| 4.1.1 Deterministische builds - 'it works on my machine' . . | 13        |
| 4.1.2 Benötigte Schnittstellen . . . . .                     | 13        |
| 4.2 Anforderungen an ein Framework . . . . .                 | 14        |
| 4.3 Einschränkungen . . . . .                                | 14        |
| 4.4 Codestruktur . . . . .                                   | 15        |

|          |                                          |           |
|----------|------------------------------------------|-----------|
| 4.4.1    | Parser für JUnit XML . . . . .           | 15        |
| 4.5      | Exemplarischer Ablauf . . . . .          | 15        |
| <b>5</b> | <b>Related Work</b>                      | <b>16</b> |
| 5.1      | Andere Tools . . . . .                   | 16        |
| <b>6</b> | <b>Fazit</b>                             | <b>17</b> |
| 6.1      | Zusammenfassung der Ergebnisse . . . . . | 17        |
| 6.2      | Möglichkeiten zur Verbesserung . . . . . | 17        |
|          | <b>Bibliography</b>                      | <b>17</b> |

# 1 Einleitung

## 1.1 Problemstellung

”One cannot “test quality into” a badly constructed software product, but neither can one build quality into a product without test and analysis.”

[Pezzè and Young, 2008, S. XVII]

Jedes größere Softwareprojekt benötigt eine Vielzahl von Tests, um die Qualität der Software zu sichern. Dabei werden die einzelnen Tests für gewöhnlich in verschiedene Klassen unterteilt, die verschiedene Abschnitte der Software abdecken. Die meisten gängigen Testframeworks bieten die Möglichkeit, einzelne Testklassen und/oder Testfälle gezielt auszuführen, ohne die restlichen Testfälle dabei mit auszuführen.

Nehmen wir z.B. einen Entwickler, der bei einem großen Softwareprojekt die Funktionalität einer Datenbank-Schnittstelle anpasst. Testfälle, welche die Funktion dieser Schnittstelle abtesten, sind hier sehr relevant - andere Testfälle, die sich mit anderen Abschnitten des Projektes befassen, wie etwa UI-Tests oder Tests für Netzwerkverbindungen, sollten bei einem sauber modularisiertem Projekt mit hoher Wahrscheinlichkeit unabhängig von Änderungen der Datenbank-Schnittstelle unverändert wie vor den Änderungen des Entwicklers ablaufen. Da manche Testfälle viel Zeit für ihre Durchführung in Anspruch nehmen, ist es daher sinnvoll, bei der Entwicklung nur einzelene relevante Abschnitte zu testen.

Dennoch muss zur Validierung des Codes regelmäßig die gesamte Testsuite durchlaufen werden, damit durch das partielle Testen des Codes keine Fehler unbemerkt im Projekt verbleiben.

Um dies zu gewährleisten, wird für gewöhnlich auf dem Build-Server noch einmal die gesamte Testsuite des Projektes durchlaufen, auch wenn manche

Testfälle bereits lokal von Entwicklern ausgeführt wurden, ohne dass sich der zu testende Code in der Zwischenzeit geändert hat. Dadurch wird die Laufzeit der gesamten CI-Pipeline unnötig verlängert.

## 1.2 Zielsetzung

Diese Bachelorarbeit untersucht, ob es möglich ist, die Laufzeit einer Testsuite (und damit der gesamten Pipeline) zu reduzieren, indem eine verlässliche Aussage darüber getroffen wird, welche Teilmenge der Testsuite für den aktuellen Stand des Codes bereits durchlaufen wurde, und welche Teilmenge noch laufen muss, um eine 100%-ige Abdeckung der Testsuite zu erreichen. Dabei soll die Wahl des Testframeworks möglichst keine Rolle spielen.

Dazu werden im Verlauf dieser Arbeit die Anforderungen an ein Tool, sowie Voraussetzungen an ein Projekt beschrieben, in welches das Tool integriert werden kann.

Im Anschluss wird evaluiert, in wie weit das Ziel der Reduktion von abzulaufenden Testcases und die damit einhergehende Reduktion der Laufzeit der gesamten Pipeline erreicht wurde.



## 2 Anforderungen

### 2.1 Anforderungen an das Tool

Folgende Anforderungen werden an einen Durchlauf des zu entwickelnden Tools gestellt:

- **Die Wahl der verwendeten Programmiersprache(n) und Testframework(s) ist unerheblich**

Das Tool soll framework-agnostisch funktionieren, auch wenn innerhalb des Projektes verschiedene Testframeworks zum Einsatz kommen.

- **Nach Ablauf des Tools wurde jeder Testcase für den aktuellen Stand des Codes genau einmal durchgeführt**

Kein Test darf mehr als genau einmal ausgeführt werden, um die Laufzeit so gering wie möglich zu halten. Trotzdem muss die gesamte Testsuite komplett abgedeckt sein, um die Aussagekraft des Testlaufes im Vergleich zu einem komplett neuen Durchlauf nicht zu beeinträchtigen.

- **Das Ergebnis der durch das Tool durchgeführten Testläufe ist am selben Ort verfügbar, wie die Ergebnisse der vorherigen Testläufe, auf die das Tool zugreift**

### 2.2 Anforderungen an ein Testausgabeformat

Jedes Testframework verfügt über eines oder mehrere mögliche Ausgabeformate, welches die Informationen über das Ergebnis eines Testlaufes enthält. Damit ein Tool entwickelt werden kann, welches möglichst frameworkunabhängig funktioniert, muss ein Ausgabeformat gefunden werden, dass von möglichst vielen verschiedenen Testframeworks auf die selbe Art und Weise

verwendet wird, damit die Dateien vom Tool über eine Schnittstelle eingelesen werden können.

Mit Blick auf diese sowie die unter Kapitel 2.1 beschriebenen Anforderungen an das Tool, ergeben sich folgende Anforderungen an ein potentielle nutzbare Ausgabeformat:

**Ein ideales Ausgabeformat für Tests:**

- **enthält die Gesamtmenge aller Tests im Projekt**

Dieser Punkt ist essenziell, zum Einen um eine Aussage darüber treffen zu können, ob eine 100%ige Testcoverage erzielt wurde, zum Anderen muss das Tool zum Erzielen einer absoluten Coverage auch auf potentiell jeden vorhandenen Testcase zugreifen können. Dazu muss dem Tool selbstverständlich auch jeder Testcase innerhalb des Projektes bekannt sein.

- **identifiziert jeden Testcase eindeutig**

Je nach Testframework und Implementierung ist es gestattet, mehreren Testcases, die in unterschiedlichen Namespaces leben, mit dem selben Namen zu bezeichnen. Aus dem Ausgabeformat muss in jedem Falle eine eindeutige Zuordnung der Testcases und den Ergebnissen möglich sein.

- **sagt für jeden Testcase einzeln aus, ob er ausgeführt wurde oder nicht**

- **ist ein strukturiertes Format**

Es muss eine erfassbare Struktur für das Testausgabeformats vorhanden sein, damit ein entsprechender Parser für das Tool entwickelt werden kann.

- **ist in vielen Frameworks über viele Programmiersprachen hin-**

**weg vertreten**

Es wäre utopisch anzunehmen, dass ein strukturiertes Format existiert, welches von jedem existierendem Testframework gleichermaßen benutzt wird. Dennoch gibt es durchaus gängige Formate, die von unterschiedlichen Testframeworks aus unterschiedlichen Sprachen unterstützt werden, wie folgend in Kapitel 3 untersucht wird.

## **2.3 Wie Output entsteht**

Hier die Pipeline Test Runner -> Test Reporter

## **2.4 git notes**

müssen nicht zwingend verwendet werden, auch andere Kommunikationswege mit Tool könnten etabliert werden. Hier nur Mittel der Wahl, weil ändern Code nicht.

Nicht zu empfehlen: Dateien im repo lagern.

### 3 Vergleich von Testausgabeformaten

Um ein Tool zu entwickeln, welches framework-agnostisch eingesetzt werden kann, muss dieses Tool die Ausgabe eines Testlaufes unabhängig vom verwendeten Framework parsen können. Die Anforderungen an ein Testausgabeformat wurden bereits in Kapitel 2.2 untersucht.

Für gewöhnlich geben Testframeworks das Ergebnis eines Testlauf in einem unstrukturiertem Format über die Konsole aus. Allerdings bieten fast alle etablierten Testframeworks auch eine Auswahl an Dateiformaten an, über welche die Testausgabe ebenfalls präsentiert werden kann.

Für diese Dateiformate wurden über die Jahre einige framework-übergreifende Ausgabeformate entwickelt, die sich in unterschiedlicher Ausprägung durchsetzen konnten.

Aufgrund der Vielzahl von verschiedenen Testframeworks und der Vielfältigkeit ihrer Ausgabeformate können im Rahmen dieser Bachelorarbeit nur die verbreitesten Formate untersucht werden.

Zur Untersuchung der praktischen Umsetzung einzelner Formate und zur exemplarischer Erfassung ihrer Verbreitung wird ein limitierter Datensatz betrachtet. Dabei gilt zu beachten, dass bei diesem Datensatz nur eine kleine Menge von Programmiersprachen beinhaltet. Außerdem beschränken sich die untersuchten Testframeworks auf die am weitesten verbreiteten in ihrer jeweiligen Sprache, um die Etablierung von Testausgabeformaten möglichst praxisnahe darstellen zu können. Außerdem bieten länger etablierte Testframeworks erfahrungsgemäß eine breitere Auswahl von Testausgabeformaten, als weniger verbreitete Testframeworks.

Tabelle 1 zeigt die Ergebnisse der Untersuchung des Datensatzes.

| Framework   | Sprache | Test-Runner | Test-Reporter | JUnit XML | XML (anderes) | OTR | TAP | JSON | HTML |
|-------------|---------|-------------|---------------|-----------|---------------|-----|-----|------|------|
| catch2      | C++     |             |               | ✓         | ✓             | ✗   | ✓   | ✗    | ✗    |
| doctest     | C++     |             |               | ✓         | ✓             | ✗   | ✗   | ✗    | ✗    |
| gtest       | C++     |             |               | ✓         | ✗             | ✗   | ✗   | ✓    | ✗    |
| hspec       | Haskell |             |               | ~         | ✗             | ✗   | ✗   | ✗    | ✗    |
| tasty       | Haskell |             |               | ~         | ✗             | ✗   | ~   | ✗    | ✗    |
| junit5      | Java    |             |               | ✓         |               |     |     |      |      |
| junit5      | Java    | maven       |               | ✓         |               | ~   |     |      | ~    |
| junit5      | Java    | gradle      |               | ✓         |               | ~   |     |      | ✓    |
| behat       | PHP     |             |               | ✓         | ✗             | ✗   | ✗   | ✓    | ✗    |
| codeception | PHP     |             |               | ✓         | ✗             | ✗   | ✗   | ✗    | ✓    |
| phpunit     | PHP     |             |               | ✓         | ✗             | ✓   | ✗   | ✗    | ✓    |
| pytest      | Python  |             |               | ✓         | ✗             | ✗   | ~   | ~    | ~    |
| unittest    | Python  |             |               | ✓         | ✗             | ✗   | ✗   | ✗    | ✗    |

✓ Nativ unterstützt  
 ~ Über Plugins unterstützt  
 ✗ Nicht offiziell unterstützt

**Tabelle 1:** Übersicht der Verbreitung bestimmter Formate

## 3.1 Evaluation der häufigsten Formate

In diesem Kapitel werden die häufigsten frameworkübergreifend verwendeten Testausgabeformate kurz vorgestellt und nach den in Kapitel 2.2 festgesetzten Kriterien auf ihre Eignung zur Benutzung als Testausgabeformat für das zu entwickelnde Tool evaluiert.

### 3.1.1 JUnit XML

#### Übersicht

JUnit XML ist ein XML-basiertes Format, welches ursprünglich für das Java-Testframework JUnit entwickelt wurde. Für JUnit XML existiert kein einheitlicher Standard, d.h. es existiert auch keine beschreibendes .xsd Schema, wie sonst für XML Formate üblich. Trotzdem wurde JUnit XML von vielen Testframeworks in gleicher oder ähnlicher Form adaptiert, wie es ur-

sprünglich für JUnit entworfen wurde. Außerdem existieren viele weitere für das Software-Testing relevante Programme, welche JUnit XML verarbeiten können, beispielsweise viele CI/CD-Systeme. Auf diese Weise hat sich JUnit XML zu einem De-Facto Standard für Ausgabeformate von Testframeworks entwickelt.

- Genaue Struktur (unterschiedliche root tags!)
- Auf unterschiedliche Umsetzungen eingehen

### **Struktur**

### **Eignung**

- Wurde speziell für Java beschrieben

## **3.1.2 Test Anything Protocol (TAP)**

### **Übersicht**

Das Test Anything Protocol wurde ursprünglich für die Programmiersprache Perl entworfen, wurde über die Jahre hinweg jedoch auch für Testframeworks in anderen Sprachen weiter entwickelt. Anders als bei JUnit XML wird für TAP ein standardisiertes Schema bereitgestellt und gepflegt (<https://testanything.org/tap-version-14-specification.html>). Anders als die meisten anderen hier untersuchten Testausgabeformate ist TAP keine spezialisierte Struktur eines bereits bestehenden Dateiformates, sondern ein eigenständiges Format.

### **Struktur**

Zu Beginn einer Datei wird die Gesamtzahl aller Testcases der Datei angegeben. Danach folgt eine Auflistung der Testergebnisse mit optionalen Zusatzinformationen, zumeist der Name des Testcase. Übersprungene Tests

können über zusätzliche Dekoratoren idikatiert werden, die in TAP als "Directives" bezeichnet werden.

Da die meisten Zusatzinformationen optional sind, zeigt Codeblock 3.1 bereits eine vollständig gültige, minimale TAP-Datei:

```
1..1
ok
```

**Codeblock 3.1:** Minimales TAP-Beispiel

Solch eine Darstellung ist natürlich ungeeignet, da keine Zuordnung zwischen Testergebnis und Testcase möglich ist. In der Praxis kann dieser Zusammenhang allerdings durch die optionalen Zusatzinformationen gesetzt werden, wie beispielhaft mit pytest in Codeblock 3.2 demonstriert:

```
# TAP results for test/pytest/test.py
ok 1 test/pytest/test.py::test_neg
not ok 2 test/pytest/test.py::test_pos
...
ok 3 test/pytest/test.py::test_skip # SKIP ...
ok 4 test/pytest/test.py::test_same_name
1..4
```

**Codeblock 3.2:** TAP, erzeugt durch pytest.  
... bezeichnet gekürzte Stellen.

## Eignung

Eine eindeutige Zuordnung zwischen Testergebnis und Testcase ist nicht in allen untersuchten Testframeworks, die TAP als Ausgabeformat bieten, möglich. Nehmen wir catch2: Zwei Testcases dürfen in diesem Framework den selben Namen aufweisen, solange sie mit unterschiedlichen Labels getaggt sind. Diese Zusatzinformationen sind allerdings nicht zwingend im TAP-Format

erforderlich, und werden vom catch2-Reporter schlicht nicht übernommen. Dies kann eine Ausgabe wie in Codeblock 3.3 zur Folge haben:

```
...
# test_same_name
not ok 3 - false
# test_same_name
ok 4 - true
...
```

**Codeblock 3.3:** TAP, erzeugt durch catch2.  
... bezeichnet gekürzte Stellen.

Dies macht TAP in diesem Falle als Ausgabeformat ungeeignet. Andere Testframeworks wie beispielsweise pytest oder tasty sorgen allerdings für eine eindeutige Zuordnung in ihren jeweiligen Ausgaben.

## Fazit

TAP ist aufgrund seiner sehr geringer Minimalstruktur und vor allem seiner nur mäßigen Verbreitung eher nicht als Testausgabeformat für das Tool geeignet.

### 3.1.3 Open Test Reporting (OTR)

#### Übersicht

Open Test Reporting (OTR) wird von dem selben Entwicklerteam, welches auch hinter dem Framework JUnit und damit auch hinter JUnit XML steht, als Nachfolger von eben diesem entwickelt. Dabei haben sie laut eigenen Angaben das Ziel, mit OTR die historisch gewachsenen Probleme von JUnit XML (siehe Kapitel 3.1.1) zu umgehen. OTR soll nach Vorstellungen der Entwickler auf lange Sicht JUnit XMLs Platz als de-facto Standard-Ausgabeformat für Testläufe einnehmen.



Anders als die anderen vorstellten Formate gibt es bei OTR zwei mögliche Dateiformate: Es gibt Implementierungen für XML und HTML. Beide Implementierungen sind im Gegensatz zu JUnit XML standardisiert und folgen einem definierten Schema. Beim XML-Format wird zwischen zwei Strukturen unterschieden: Hierarchisch und eventbasiert. Beide Formate nutzen das selbe Set von Kernelementen, unterscheiden sich aber in ihrer Struktur. Für beide Strukturen gibt es Programmiersprachen-spezifische Erweiterungen. Das HTML-Format wird ausgehend von einer vorher erzeugten OTR-XML Datei erzeugt, und ist ebenso erweiterbar.

## **Struktur**

### **Eignung**

Da OTR zum Zeitpunkt der Niederschrift dieser Bachelorarbeit noch sehr jung ist und aktiv entwickelt wird, bleibt abzuwarten, ob die Entwickler ihr Ziel eines standardisierten, Programmiersprachen-agnostischen Testausgabeformates erreichen können.

Außerdem ist die Verbreitung von OTR noch sehr limitiert: Von den in Tabelle 1 betrachteten Testframeworks ist lediglich phpunit in der Lage, OTR nativ zu erzeugen. Außerdem kann junit5 (welches, wie eingangs erwähnt, vom selben Entwicklerteam wie OTR betreut wird) zumindest über Plugins eine OTR-Ausgabe erzeugen.

### **3.1.4 Common Test Report Format (CTRF)**

#### **Übersicht**

Ähnlich wie JUnit XML ein spezielles Format für das Dateiformat XML ist, so gibt es auch ein spezielles Testausgabeformat für JSON: Das Common Test Report Format, kurz CTRF. Ähnlich wie OTR ist CTRF zum aktuellen Zeitpunkt ein noch sehr junges Format, welches aktiv weiterentwickelt wird.

Dies zeigt sich auch in seiner Verbreitung: Keine der in Tabelle 1 untersuchten Testframeworks bieten die Möglichkeit, CTRF zu erzeugen, weder nativ noch über ein offizielles Plugin.

CTRF ist dennoch interessant zu untersuchen, da es zum Einen das einzige größere frameworkagnostische JSON-Format ist, und zum Anderen, da es striktere Vorgaben für seine Struktur als die anderen zuvor untersuchten Testausgabeformate macht.

## **Struktur**

Was CTRF aus den übrigen Formaten herausstechen lässt, ist die Tatsache, dass ein Name für Testcases in der Ausgabe zwingend erforderlich ist.

## **Eignung**

### **3.1.5 Andere Formate**

teamcity, testdox?

Neben den bisher untersuchten ganz oder zeilweise strukturierten Formaten

Selbes Problem wie JSON. Beispielsweise HTML, CSV, Markdown etc. Nicht strukturierte Ausgaben (pretty etc.) werden nicht betrachtet.

Bezieht sich nur auf über die offiziellen von den jeweiligen package managern bereitgestellte Formate. Es gibt viele weitere Plugins im unabhängigen Bereich.

## **3.2 Fazit**

Anhand der Evaluation der unterschiedlichen Ausgabeformate in Kapitel 3 wird deutlich, dass es keinen eindeutigen Kandidaten für die Auswahl des Testausgabeformats, welches vom Tool als Input akzeptiert wird, gibt. Ein gemeinsames Problem: zeigen keine nicht-ausgeführten tests an

## 4 Umsetzung

### 4.1 Anforderungen an ein Projekt

#### 4.1.1 Deterministische builds - 'it works on my machine'

Damit das Tool eine qualifizierte Aussage über die Testergebnisse treffen kann, muss gewährleistet sein, dass alle Tests unabhängig von der Entwicklungsumgebung immer das selbe Testergebnis liefern. Ein Testfall, der auf der Machine eines Entwicklers ein bestimmtes Ergebnis liefert, muss auch zwingend auf einem Build-Server das selbe Ergebnis liefern. Andernfalls könnte eine Diskrepanz zwischen den Testergebnissen zu einer verfälschten Aussage des Tools über den aktuellen Stand der Test führen.

Da Junit XML nicht alle in Kapitel 2.2 aufgeführten Anforderungen erfüllt, ergeben sich einige weitere Anforderungen an ein Projekt, das das Tool nutzen möchte, um trotzdem das geforderte Ziel zu erreichen.

#### 4.1.2 Benötigte Schnittstellen

##### **test discovery**

Damit das Tool die Gesamtmenge aller Tests im Projekt kennt, muss im Projekt ein Script vorhanden sein, welches die die Gesamtmenge aller Tests in einem spezifiziertem Format ausgibt.

-> qualifiedname: Eine Zeichenfolge, die einen Test innerhalb eines Testframeworks eindeutig identifiziert. Beispiele in vers. TestFrameworks bringen!

-> Hier Format beschreiben/definieren

### **test execution**

Zum Anderen muss dem Tool ein zweites Script zur Verfügung gestellt werden, welches eine Liste von zuvor in Kapitel 4.1.2 beschriebenen qualifiedNames entgegennimmt, nur diese Tests ausführt, und das Ergebnis (auch wieder JUnit XML) an das Tool zurückmeldet, welches dann diesen Input zusätzlich an die gitnotes anhängt.

Absehen von diesen Anforderungen muss dem Tool lediglich der Pfad zu einem Ordner, der alle relevanten JUnit-XML Dateien enthält, mitgegeben werden.

Zur Vereinfachung dieses Prozesses benutzt das Tool die Datei "config.json", in der die Argumente zum Aufrufen der beiden Skripten, sowie der Pfad zu dem Ordner mit den JUnit-XML Dateien hinterlegt werden.

-> Hier Diagram mit Interaktion zwischen Skripten/Tool einfügen

## **4.2 Anforderungen an ein Framework**

- kann JUnit XML
- enthält essenzielle Informationen für einen Test (genau ausführen!)
- kann tests einzeln aufrufen

## **4.3 Einschränkungen**

- Annahme: keine deliberate bad actors; keine Authentifizierung
- Annahme: An gitnotes angehängte Dateien spiegeln aktuellen Stand vom Code wieder, keine Veränderung der Codebase im Nachhinein

## **4.4 Codestruktur**

### **4.4.1 Parser für JUnit XML**

- Parser des tools - leerzeilen als standart-separator von gitnotes

## **4.5 Exemplarischer Ablauf**

`git notes add -F "$(realpath code/out/report.xml)"HEAD`

## 5 Related Work

### 5.1 Andere Tools

Abgrenzung zu z.B Bazel/Gradle?

## 6 Fazit

### 6.1 Zusammenfassung der Ergebnisse

### 6.2 Möglichkeiten zur Verbesserung

In seiner aktuellen Form kann das Tool nicht entscheiden, ob die XML-Ausgabe der Tests auch tatsächlich aus dem aktuellen Stand des Codes entstammt, oder ob im Nachhinein Änderungen an dem zu testenden Code oder den Tests selber vorgenommen wurden, was die Aussagekraft des Tools stark beeinträchtigt.

Ein anderes Problem ist, dass das Tool alle Tests, die nicht an den letzten Schwung gitnotes anhängt sind, als nicht abgelaufen betrachtet, auch wenn einige Testfälle in einer vorherigen Iteration ausgeführt sein könnten, und die Änderungen, die seither im Code gemacht wurden diese Tests überhaupt nicht betreffen.

Eine Möglichkeit, dieses Problem zu beheben, wäre die Integration eines neuen Tools, welches eine Seletive Regression Testing / Regression Test Selection implementiert, und so dem Tool rückmelden kann, welche Tests tatsächlich neu ausgeführt werden müssen, und bei welchen der vorherige Stand noch gültig ist.

Zusätzliche Unterstützung von TAP wäre realistisch, JSON formate eher unrealistisch.

-> Ekstazi <https://github.com/gliga/ekstazi>

# Literaturverzeichnis

Mauro Pezzè and Michal Young. *Software Testing and Analysis: Process, Principles, and Techniques*. 2008.



